

C2_01 Advanced Excel & Power Query

Astra Parity RC4 Learner Book

Ten lessons for auditable formulas, dynamic summaries, repeatable Power Query imports, safe merges/appends, parameters, and refreshable month-end reporting.

Route: Learner Book -> matching practice -> Mini-Lab -> Quick Revision -> non-gating Module Test -> protected review -> Gate A. Gate B/C remain fresh retests only when assigned.

Module map

ID	Sub
AX1	Advanced formulas without formula spaghetti
AX2	XLOOKUP, INDEX/MATCH and lookup design
AX3	Dynamic arrays: FILTER, UNIQUE, SORT and spill-safe workflows
AX4	SUMIFS, COUNTIFS and auditable conditional KPIs
AX5	Advanced PivotTables and business summaries
AX6	Power Query architecture and repeatable imports
AX7	Cleaning and transforming multiple files
AX8	Merge vs append and relationship/grain mistakes
AX9	Parameters and reusable query design
AX10	Refreshable monthly reporting workflow

Release discipline: A refresh that runs is not enough. You must prove file scope, keys, row counts, totals, exceptions and business logic before calling the output safe.

AX1 - Advanced formulas without formula spaghetti

1. Why it matters

A workbook becomes risky when business rules are copied into many long formulas. One rule change can leave half the workbook using the old value and half using the new value.

2. What you are learning

Build a layered workbook: raw inputs, visible assumptions, calculations, outputs and controls. Use structured references or named cells so business constants live in one place.

3. Prerequisite / what you already know

Course 1 formulas, cell references, Excel Tables, IF/SUM basics. If structured references are unfamiliar, read the example slowly; they are defined here.

4. Picture it in your head

Calculation layers: Picture a complex formula as a stack of transparent sheets. One sheet cleans inputs, one applies business logic, one returns the presentation value. If you cannot inspect each layer separately, the formula has become spaghetti.

5. New words before we use them

Structured reference: A formula reference that uses Excel Table and column names, such as Sales[Amount], instead of fixed coordinates. Assumption cell:

One visible cell that stores a business constant such as a tax rate, target, or threshold. Control formula: A formula designed to expose an error or mismatch rather than produce the business output.

6. Watch / visual source

RECOMMENDED VIDEO: Best Excel Formulas in an hour - Beginner to PRO Masterclass 1:51-8:28 for conditional summaries; 1:01:41 for report-with-formulas context.

https://www.youtube.com/watch?v=dVJFyql_Nw4

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Create an Excel Table named tSales with columns OrderID, Gross, DiscountRate and Net.
- Put the default discount in a visible assumptions cell named pDefaultDiscount instead of typing 0.05 into many formulas.
- Use a readable formula pattern. Example: `=[@Gross]*(1-IF([@DiscountRate]="",pDefaultDiscount,[@DiscountRate]))`.
- Create a summary output with `=SUM(tSales[Net])`. Then create a separate control comparing that total to an independently calculated
- gross-minus-discount total.
- Add an invariant check: no row may have Net > Gross. A formula can calculate `=COUNTIFS(tSales[Net], ">"&tSales[Gross])` only in versions
- that support this exact pattern reliably; otherwise use a helper flag and COUNTIF. The principle is the control, not memorizing one clever
- formula.

8. What you should see / be able to say

Business constants are visible, formulas are shorter, and a reviewer can trace input -> rule -> calculation -> output -> check.

9. Why it worked

One assumption is changed once and flows through the calculation. The control is separate from the output, so it can fail visibly. Professional upgrade:

Professional Excel files make business rules discoverable. Use Tables, named assumptions, clear calculation columns, comments/notes where needed, and controls that a future owner can rerun.

10. Common beginner mistakes

1. Hard-coding the same percentage in many formulas. | 2. Building one giant nested formula because it looks advanced. | 3. Mixing raw data and manual overrides inside the calculation column. | 4. Checking only that the formula returns a number.

11. If you get stuck - recovery ladder

Split the formula into pieces. First verify the row input, then the assumption cell, then the calculation, then the summary, then the independent control.

Do not debug all layers at once. If still stuck, use only the partial skeleton in the practice workbook; save the attempt before opening review.

12. Now you do a small version alone

Fresh case: create a shipping-cost workbook where FuelSurcharge and FreeShippingThreshold are visible assumptions. Produce line cost, order total and two controls.

13. Validate your result

Change the surcharge once and confirm only intended outputs change. Reconcile line totals to the summary and test one impossible-value rule.

14. Teach it back in your own words

Explain why fewer formulas can be more professional than a more complicated formula.

15. Protected review + changed fresh retry

Save your independent workbook first. Then open AX1 review, close it, and rebuild with a changed fee rule without copying the finished formula.

16. Evidence / mastery + exact next route

Workbook screenshot/file + assumption block + two controls + 60-90 second explain-back. After saving evidence, continue to AX2.

AX2 - XLOOKUP, INDEX/MATCH and lookup design

1. Why it matters

A lookup can return a plausible value while the lookup table contains duplicate keys, missing keys, or the wrong business identifier. The formula may be syntactically correct and still be analytically unsafe.

2. What you are learning

Design a lookup contract: business key, lookup range, return field, exact-match behavior, no-match handling, duplicate-key test and unmatched-key reconciliation. Use INDEX/MATCH as a transferable fallback mental model.

3. Prerequisite / what you already know

AX1 formula architecture; Course 1 exact-match lookup awareness. Modern XLOOKUP requires a supported Excel version.

4. Picture it in your head

Key-to-value map: A lookup is a controlled question: "for this key, which one value should I retrieve from which table?" First prove key uniqueness and match behavior; only then choose XLOOKUP or INDEX/MATCH syntax.

5. New words before we use them

Business key: A column or column combination intended to identify the business entity used for matching. Exact match: Return only when the lookup key equals the target key according to the chosen matching rule. Unmatched key: A source key for which the lookup table has no valid matching record.

6. Watch / visual source

Lookup contract + dirty-key check + unmatched list + reconciled row count + explain-back. REQUIRED VISUAL SUPPORT: 10 Advanced XLOOKUP Tips &

Tricks 1:15-4:18 for basic XLOOKUP and wildcard behavior; use later tricks only as enrichment.

<https://www.youtube.com/watch?v=YyhJe5tMq58>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Before writing XLOOKUP, test the product master: ProductID should be unique and nonblank.
- Use visible no-match output: =XLOOKUP([@ProductID],tProduct[ProductID],tProduct[Category],"UNMATCHED").
- Count UNMATCHED results before replacing or hiding them. Investigate whether the source key is new, mistyped, or missing from the master.
- Fallback mental model: MATCH finds the row position of the key; INDEX returns a value from the return column at that position. The business
- contract is the same even when the function changes.
- Reconcile row count: a lookup should enrich each source row; it should not silently add or remove source rows.

8. What you should see / be able to say

Every source row has one trusted category or a visible UNMATCHED exception, and duplicate lookup keys are investigated rather than ignored.

9. Why it worked

The key-quality check happens before the retrieval, so a successful formula cannot hide a broken master table.

Professional upgrade: Use lookup formulas

for appropriate spreadsheet-scale enrichment, but preserve key-quality and unmatched controls. For repeatable larger transformations, the same reasoning

transfers to Power Query merges and database joins.

10. Common beginner mistakes

1. Wrapping IFERROR around everything before inspecting the cause. | 2. Using approximate match accidentally. | 3. Assuming a code is unique because it looks like an ID. | 4. Choosing XLOOKUP only because it is newer, not because the contract is clear.

11. If you get stuck - recovery ladder

Run three checks in order: blank key count, duplicate key count in master, unmatched source key count. Repair the data contract before changing the formula. If still stuck, use only the partial skeleton in the practice workbook; save the attempt before opening review.

12. Now you do a small version alone

Fresh case: attach ServiceTier from an Account master to ticket rows. The dirty master contains one duplicated AccountID and two source IDs not in the master. Build visible exception handling.

13. Validate your result

Source row count unchanged; master business key unique after repair; unmatched count explained; spot-check at least three IDs.

14. Teach it back in your own words

Explain why XLOOKUP does not protect you from a bad business key.

15. Protected review + changed fresh retry

Attempt first. After review, use a changed employee-to-department lookup with different no-match behavior.

16. Evidence / mastery + exact next route

Lookup contract + dirty-key check + unmatched list + reconciled row count + explain-back. REQUIRED VISUAL SUPPORT: 10 Advanced XLOOKUP

Tips & Tricks 1:15-4:18 for basic XLOOKUP and wildcard behavior; use later tricks only as enrichment.

<https://www.youtube.com/watch?v=YyhJe5tMq58> After saving evidence, continue to AX3.

AX3 - Dynamic arrays: FILTER, UNIQUE, SORT and spill-safe workflows

1. Why it matters

Dynamic arrays make analyst views fast, but a blocked spill range or unsupported workbook version can make a correct-looking design fail.

You must understand the returned array, not just copy a formula.

2. What you are learning

Use FILTER, UNIQUE and SORT to create live analysis views; understand spill ranges, #SPILL!, source expansion and version limits; validate returned row counts independently.

3. Prerequisite / what you already know

AX1 Tables and formulas. Requires a modern Excel version supporting dynamic arrays for hands-on mastery.

4. Picture it in your head

Spill zone: Dynamic arrays are a result that needs room to breathe. One formula can return many rows/columns, so picture a reserved spill rectangle. Anything blocking that rectangle is a design conflict, not a reason to copy formulas down.

5. New words before we use them

Dynamic array: A formula result that can return multiple cells from one formula. Spill range: The cells automatically occupied by a dynamic-array result.

#SPILL!: An error shown when Excel cannot place the full dynamic-array result into its intended cells.

6. Watch / visual source

RECOMMENDED VIDEO: Best Excel Formulas in an hour - Beginner to PRO Masterclass 8:28-12:21 FILTER; 33:44 UNIQUE; 57:22 FILTER + dates.

https://www.youtube.com/watch?v=dVJFyqI_Nw4

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- With tSales as the source, create =SORT(UNIQUE(tSales[Region])) to generate a live region list.
- Create a separate view: =FILTER(tSales[[OrderID]:[Net]],tSales[Status]="Open","No open orders").
- Predict the expected row count using COUNTIF before trusting the spill.
- Type a value into one cell where the result needs to spill. Observe #SPILL!, inspect the obstruction, remove it, and rerun the row-count control.
- Add a new row to the source Table and confirm the dynamic result expands automatically.

8. What you should see / be able to say

A live spill-safe analysis view that expands with the source and has an independent count control.

9. Why it worked

The Table handles source expansion; the dynamic-array formula handles output expansion; the independent count checks completeness. Professional

upgrade: Dynamic arrays are excellent for QA lists, lightweight reporting and helper views. For governed repeatable transformations, decide whether Power

Query or the data model is the more maintainable layer.

10. Common beginner mistakes

1. Putting spill formulas inside a Table calculated column. | 2. Leaving manual notes inside a spill range. | 3. Ignoring workbook/version compatibility. | 4. Assuming a visually correct list contains every eligible row.

11. If you get stuck - recovery ladder

Select the formula cell, inspect the intended spill border/error, clear obstructions, verify the source Table range, then compare the returned count to COUNTIF/COUNTIFS. If still stuck, use only the partial skeleton in the practice workbook; save the attempt before opening review.

12. Now you do a small version alone

Fresh case: build a sorted unique owner list and a live FILTER view of unresolved priority tickets. Prove count parity.

13. Validate your result

Independent eligible-row count equals the spill output count; source expansion works; no manual data sits inside spill range.

14. Teach it back in your own words

Explain the difference between one formula cell and its spilled result.

15. Protected review + changed fresh retry

Save attempt, then review. Changed retry: a product/region exception list with a different condition and sort order.

16. Evidence / mastery + exact next route

Working dynamic-array view + row-count control + #SPILL! recovery evidence + explain-back. After saving evidence, continue to AX4.

AX4 - SUMIFS, COUNTIFS and auditable conditional KPIs

1. Why it matters

Conditional formulas are easy to write and easy to mis-scope. One wrong date boundary or denominator can produce a convincing KPI that answers a different question.

2. What you are learning

Translate a metric contract into SUMIFS/COUNTIFS criteria; separate numerator and denominator; use real dates; reconcile conditional summaries to independent totals.

3. Prerequisite / what you already know

SAF4 metric contracts from C2-00 and AX1 formula architecture.

4. Picture it in your head

Filter gates: SUMIFS/COUNTIFS send rows through several gates - region, date, status, product, etc. A row contributes only if it passes every required gate. Write each criterion as a business rule before writing the formula.

5. New words before we use them

Criteria range: The range evaluated against a condition in SUMIFS/COUNTIFS. Numerator: The count or amount on top of a rate. Denominator: The eligible population underneath a rate.

6. Watch / visual source

RECOMMENDED VIDEO: Best Excel Formulas in an hour - Beginner to PRO Masterclass 1:51-8:28

SUMIFS/COUNTIFS/AVERAGEIFS; 1:01:41
report-with-formulas.

https://www.youtube.com/watch?v=dVJFyql_Nw4

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Define period first: use start date $\geq$ first day and date $<$ first day of next month to avoid time-of-day edge cases.
- Completed revenue pattern:
 - `=SUMIFS(tSales[Net],tSales[Region],$A2,tSales[Status],"Completed",tSales[OrderDate], ">="&B$1,tSales[OrderDate], "<"&EDATE(B$1,1`
- Build numerator and denominator in separate cells before dividing. Example numerator = completed orders; denominator = eligible non-test
- orders.
- Use `=IFERROR(Numerator/Denominator,"")` only after denominator quality is understood; do not use IFERROR to hide a zero-population
- definition problem.
- Reconcile all region totals back to an independently filtered/pivoted source total.

8. What you should see / be able to say

A KPI grid with explicit criteria, visible numerator/denominator and a grand-total reconciliation.

9. Why it worked

The metric definition is converted into auditable criteria one field at a time, and the period uses an exclusive upper boundary. Professional upgrade: Keep

metric logic consistent across spreadsheet, Power BI and SQL layers. A professional handoff includes the metric contract next to the formula design.

10. Common beginner mistakes

1. Using text months instead of real dates. | 2. Forgetting one criterion on the denominator. | 3. Using SUMIF/SUMIFS totals as their own validation. | 4. Hiding zero denominators without explaining why they are zero.

11. If you get stuck - recovery ladder

Write the KPI in English. Highlight population, event/status, date basis and segment. Map each phrase to exactly one formula criterion, then test criteria one at a time. If still stuck, use only the partial skeleton in the practice workbook; save the attempt before opening review.

12. Now you do a small version alone

Fresh case: build On-Time Delivery Rate by region for three months with visible numerator/denominator and a control total.

13. Validate your result

Manually filter one region/month, compare numerator and denominator, then reconcile grid totals to source.

14. Teach it back in your own words

Explain why a rate is usually harder to define than a simple count.

15. Protected review + changed fresh retry

Attempt first. Changed retry uses refund rate with a different eligible population.

16. Evidence / mastery + exact next route

KPI contract + conditional formulas + independent reconciliation + explain-back. After saving evidence, continue to AX5.

AX5 - Advanced PivotTables and business summaries

1. Why it matters

PivotTables can answer business questions quickly, but they will happily count text when you expected a sum, summarize duplicated data, or stay stale when the source changes.

2. What you are learning

Create a PivotTable from a clean Table; choose Rows/Columns/Values/Filters intentionally; change aggregation; group dates; use slicers; refresh and reconcile grand totals.

3. Prerequisite / what you already know

Course 1 basic PivotTable exposure, AX4 metric definitions, clean tabular source.

4. Picture it in your head

Summary cube: A PivotTable takes one clean fact table and lets you rotate the question: rows, columns, filters and values. The underlying grain does not change just because the summary view changes.

5. New words before we use them

Values aggregation: The summary operation applied in the Values area, such as Sum, Count or Average. Slicer: A clickable visual filter connected to a PivotTable. Pivot cache: Excel's internal stored representation used by PivotTables; refresh is needed when source data changes.

6. Watch / visual source

Pivot file + business question + aggregation rationale + refresh proof + reconciliation. REQUIRED VISUAL SUPPORT: Excel Pivot Tables - Master Data Analysis in just 45 minutes 0:51-24:40 required core; 31:43+ optional enrichment.

<https://www.youtube.com/watch?v=zuSNd1ZMfBI>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Start from one Excel Table with one header row and no subtotal rows.
- Build a PivotTable: Customer in Rows; CallID in Values. Confirm whether Excel chose Count or Sum and why.
- Add Satisfaction to Columns or Values depending on the question; do not treat categories as numeric measures.
- Add Date and group to month only after confirming the source column contains real dates.
- Record the source control total and compare it to Pivot grand total. Add one new source row, refresh, and prove the Pivot changed.

8. What you should see / be able to say

A PivotTable whose field choices answer a stated question, uses the correct aggregation, refreshes, and reconciles to source.

9. Why it worked

The question drives the field layout; the source Table controls expansion; the reconciliation catches stale or duplicated inputs. Professional upgrade: Use PivotTables for rapid exploration, QA and stakeholder working sessions. Move stable repeatable transformations upstream and document any calculated fields/measures.

10. Common beginner mistakes

1. Dragging fields until the table looks interesting without stating the question. | 2. Trusting Excel's default Sum/Count. | 3. Building pivots from ranges that exclude new rows. | 4. Using a PivotTable to hide source data-quality problems.

11. If you get stuck - recovery ladder

Return to the source Table. Confirm grain, field data types and control total. Rebuild the smallest pivot with one Row and one Value, verify aggregation, then add complexity. If still stuck, use only the partial skeleton in the practice workbook; save the attempt before opening review.

12. Now you do a small version alone

Fresh case: analyze support tickets by team, month and priority. Create one slicer and one trend PivotChart. Reconcile total ticket count.

13. Validate your result

Source row count/grain documented; Pivot grand total reconciles; refresh test changes as expected; aggregation rationale saved.

14. Teach it back in your own words

Explain why a PivotTable is a question-answering layer, not a data-cleaning layer.

15. Protected review + changed fresh retry

Attempt first. Changed retry uses orders by category and channel with a different value aggregation.

16. Evidence / mastery + exact next route

Pivot file + business question + aggregation rationale + refresh proof + reconciliation. REQUIRED VISUAL SUPPORT: Excel Pivot Tables - Master Data

Analysis in just 45 minutes 0:51-24:40 required core; 31:43+ optional enrichment.

<https://www.youtube.com/watch?v=zuSNd1ZMfBI> After saving evidence, continue to AX6.

AX6 - Power Query architecture and repeatable imports

1. Why it matters

Manual copy/paste cleaning must be repeated every month and is hard to audit. Power Query stores a repeatable sequence of transformations-but a successful refresh still needs controls.

2. What you are learning

Understand the Power Query Editor, queries, Applied Steps, source, types, transformations, load destination and refresh. Build steps that are readable and predictable.

3. Prerequisite / what you already know

Basic Excel tables/files. No M language knowledge is required; M is introduced only as the code behind Applied Steps.

4. Picture it in your head

Query assembly line: Source -> promote/types -> clean -> standardize -> validate -> load. Each step should have one clear purpose and a stable name. If a source changes, walk the line until you find the first broken contract.

5. New words before we use them

Query: A repeatable set of instructions that connects to and transforms data. Applied Step: One recorded transformation in a Power Query query. M: The Power Query formula language generated behind many UI actions. Load: Sending the query result to a worksheet, Data Model or another supported destination.

6. Watch / visual source

Excel file with query + Applied Steps screenshot + row/total controls + explain-back. REQUIRED VIDEO: Power Query - Beginner to PRO Masterclass in 30 minutes 4:54-13:36 editor/clean/load; 17:50-28:25 business cleaning; 33:35-35:17 load/refresh.
<https://www.youtube.com/watch?v=MMdcczmULrU>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Use Data -> Get Data -> From Text/CSV (exact ribbon wording can vary by Excel build), then choose Transform Data.
- In Power Query Editor, identify Queries pane, data preview, Applied Steps and formula bar. Rename the query qSales_Staging.
- Set OrderDate to Date and Amount to Decimal Number deliberately. Trim text status values and rename unclear columns.
- Before filtering, record source row count/control total. Apply a filter and state why row count should change.
- Rename important steps so a future analyst can read the pipeline. Load to a worksheet Table only after validation.

8. What you should see / be able to say

A refreshable query with readable steps, explicit data types, known row-count changes and a documented output.

9. Why it worked

Transformation logic is stored once and reruns when the source changes; controls make refresh success different from data correctness. Professional upgrade: Organize queries by stage (staging/reference/output), filter early when appropriate, postpone expensive operations, set types deliberately, and document complex steps.

10. Common beginner mistakes

1. Cleaning manually before Power Query and losing reproducibility. | 2. Accepting automatically inferred types without checking. | 3. Deleting error rows before investigating why they are errors. | 4. Using dozens of anonymous steps with no logic grouping.

11. If you get stuck - recovery ladder

Select Applied Steps from top to bottom. Find the first step where row count/type/value becomes wrong. Fix or move that step; do not restart the entire query blindly. If still stuck, use only the partial skeleton in the practice workbook; save the attempt before opening review.

12. Now you do a small version alone

Fresh case: import a customer CSV, standardize dates/text, create one conditional column, and save before/after controls.

13. Validate your result

Source row count known; expected row-changing steps explained; types correct; refresh reproduces same result.

14. Teach it back in your own words

Explain Power Query as instructions, not as a cleaned copy of data.

15. Protected review + changed fresh retry

Attempt first. Changed retry uses a service log with a different date/text problem.

16. Evidence / mastery + exact next route

Excel file with query + Applied Steps screenshot + row/total controls + explain-back. REQUIRED VIDEO: Power Query - Beginner to PRO Masterclass

in 30 minutes 4:54-13:36 editor/clean/load; 17:50-28:25 business cleaning; 33:35-35:17 load/refresh.

<https://www.youtube.com/watch?v=MMdcczmULrU>

After saving evidence, continue to AX7.

AX7 - Cleaning and transforming multiple files

1. Why it matters

Folder combine can automate monthly ingestion, but it can also silently ingest backup files, omit a month, or break when one file's schema drifts.

2. What you are learning

Use From Folder; filter intended files before combining; understand sample-file transformation; preserve Source.Name; validate file count, row contribution, schema and totals; diagnose drift.

3. Prerequisite / what you already know

AX6 single-query workflow. Comfortable with CSV files and expected column names.

4. Picture it in your head

Folder intake dock: Treat a folder as a controlled loading dock, not a bucket. Approved files enter; backups/temp files are rejected; schema is checked; Source.Name is kept so every row can be traced back to its file.

5. New words before we use them

Folder connector: A Power Query connector that lists files from a folder or file-system-like source. Sample file: The representative file Power Query uses to define the transformation applied to combined files. Schema drift: A change to expected columns/types/structure between source files. File contribution: The rows or totals contributed by each source file to the combined output.

6. Watch / visual source

Folder query + expected/actual file list + per-file contributions + drift recovery + explain-back. REQUIRED VIDEO: Power Query - Beginner to PRO Masterclass in 30 minutes Use 4:54-13:36 for editor flow; course lab teaches folder-specific controls. <https://www.youtube.com/watch?v=MMdcczmULrU>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Point From Folder to a dedicated practice folder. Inspect the file list before pressing Combine.
- Filter Extension = .csv and explicitly exclude names containing backup/IGNORE. Do this before combine.
- Combine only intended monthly files. Keep Source.Name in the output so every row retains file lineage.
- Build a contribution table: Source.Name, row count and Net total. Expect exactly three clean monthly files.
- Add the provided April drift file only after the clean run. Diagnose the changed column/type instead of deleting the failing file.

8. What you should see / be able to say

One combined table with source lineage and an auditable per-file contribution check; drift creates a visible exception, not silent omission.

9. Why it worked

File scope is controlled before combine and Source.Name remains available for reconciliation after combine.

Professional upgrade: Production folder

ingestion should define filename/scope rules, expected cadence, schema contract, file contribution controls and failure handling before refresh is considered releasable.

10. Common beginner mistakes

1. Using Combine while backup files are in the same folder. | 2. Deleting Source.Name too early. | 3. Fixing one monthly file manually outside the pipeline. | 4. Treating a successful refresh as proof that all expected months arrived.

11. If you get stuck - recovery ladder

Return to the folder listing query. Compare actual filenames to expected filenames, then inspect sample transformation and the first failing file/column.

Repair the structural cause. If still stuck, use only the partial skeleton in the practice workbook; save the attempt before opening review.

12. Now you do a small version alone

Fresh case: combine three service CSVs, deliberately introduce one backup and one drift file, then create file-count and contribution controls.

13. Validate your result

Expected vs actual file list; row counts by file; total by file; schema exception documented; no backup contribution.

14. Teach it back in your own words

Explain why Source.Name is a data-quality field during ingestion.

15. Protected review + changed fresh retry

Attempt first. Changed retry uses a different folder with one missing expected month and one unexpected extra file.

16. Evidence / mastery + exact next route

Folder query + expected/actual file list + per-file contributions + drift recovery + explain-back. REQUIRED VIDEO:

Power Query - Beginner to PRO

Masterclass in 30 minutes Use 4:54-13:36 for editor flow; course lab teaches folder-specific controls.

<https://www.youtube.com/watch?v=MMdcczmULrU>

After saving evidence, continue to AX8.

AX8 - Merge vs append and relationship/grain mistakes

1. Why it matters

Append and merge solve different problems. Choosing the wrong one-or expanding a merge against duplicate dimension keys-can multiply records and totals without an obvious error.

2. What you are learning

Append compatible rows; merge columns by keys; declare input/output grain; verify right-side key uniqueness; use join kinds deliberately; reconcile rows/totals before and after.

3. Prerequisite / what you already know

SAF5 grain reasoning, AX6 Power Query basics, AX7 file lineage.

4. Picture it in your head

Stack vs enrich: Append stacks rows with the same meaning. Merge enriches rows using a key. If you picture the operation first, you avoid using a merge where an append belongs or multiplying rows with a dirty key.

5. New words before we use them

Append: Stack the rows of tables into one combined table by column names. Merge: Join columns from two queries using matching key values. Join kind: Rule defining which matched/unmatched rows are retained, such as Left Outer or Inner. Left anti: Rows from the left table that have no match in the right table.

6. Watch / visual source

RECOMMENDED VIDEO: Power Query - Beginner to PRO Masterclass in 30 minutes Visual editor support only; precise merge/append semantics use Microsoft references.

<https://www.youtube.com/watch?v=MMdcczmULrU>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Declare both monthly sales tables as one row per order line with compatible columns. Append them as new.
- Declare Product Master as one row per ProductID. Test uniqueness before merge.
- Merge sales (left) to product master (right) on ProductID using Left Outer when every sales row must remain visible.
- Before expanding, inspect match behavior. Expand only needed attributes. Create a separate Left Anti query to list unmatched ProductIDs.
- Reconcile sales row count and additive Net total before/after merge. If row count increases, inspect duplicate right-side keys before doing
- anything else.

8. What you should see / be able to say

Append increases rows by the sum of source periods; merge enriches rows without multiplying the fact grain; exceptions stay visible.

9. Why it worked

The grain contract predicts what row count should do, turning unexpected multiplication into an immediate failure signal. Professional upgrade: Power Query merges should carry the same controls as SQL joins and BI relationships: key uniqueness, cardinality expectation, unmatched handling and before/after

reconciliation.

10. Common beginner mistakes

1. Appending a dimension to a fact table. | 2. Merging on a descriptive name instead of a stable key without justification. | 3. Expanding a many-match table and accepting row multiplication. | 4. Using Remove Duplicates after a bad merge to hide the symptom.

11. If you get stuck - recovery ladder

Write left grain, right grain and expected output grain. Check right-key uniqueness. Run left anti. Compare before/after row count and total. Only then revise the join. If still stuck, use only the partial skeleton in the practice workbook; save the attempt before opening review.

12. Now you do a small version alone

Fresh case: append two ticket months and merge Team metadata. The dirty Team master includes a duplicate TeamID. Diagnose and repair without DISTINCT/remove-duplicates as a blind fix.

13. Validate your result

Append row arithmetic correct; right key unique; unmatched list explained; fact row count and total stable after merge.

14. Teach it back in your own words

Explain append vs merge using 'more rows' versus 'more columns'-then explain why that shortcut is not enough without grain.

15. Protected review + changed fresh retry

Attempt first. Changed retry uses customer master with one missing key and a different join kind decision.

16. Evidence / mastery + exact next route

Grain contract + append/merge queries + anti-join exception + reconciliation + explain-back. After saving evidence, continue to AX9.

AX9 - Parameters and reusable query design

1. Why it matters

A file path, reporting date or environment value copied into several queries is easy to change inconsistently. A parameter centralizes configuration-but it is not a place to hide secrets or business definitions.

2. What you are learning

Create a parameter contract; define purpose/type/current value/allowed values/owner; reference it in a query; change one value and prove only the intended data changes.

3. Prerequisite / what you already know

AX6 query basics. Understand that a parameter is configuration, not source data.

4. Picture it in your head

Control knob: A parameter is a labeled knob that changes a controlled input - folder path, date cutoff, threshold - without rewriting transformation logic. The query should behave predictably for every allowed knob position.

5. New words before we use them

Parameter: A named value that can be reused in query logic or data-source functions. Configuration: A controlled setting that changes how the process runs without rewriting the core logic. Allowed value: A permitted parameter value or range used to reduce accidental misuse.

6. Watch / visual source

RECOMMENDED VIDEO: Power Query - Beginner to PRO Masterclass in 30 minutes Editor context only; parameter contract is course-led.

<https://www.youtube.com/watch?v=MMdcczmULrU>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Create pStartDate as type Date and document why it exists, who changes it and what it should affect.
- Reference pStartDate in the query filter instead of typing a literal date into several steps.
- Before changing it, record current row count and total. Change only pStartDate, refresh, and predict direction of row-count change.
- Compare before/after results. If unrelated fields change, inspect query dependencies.
- Never store passwords/API secrets as ordinary worksheet parameters. Use appropriate credential mechanisms for real connections.

8. What you should see / be able to say

One controlled configuration value changes the intended query behavior and leaves unrelated logic unchanged.

9. Why it worked

The parameter separates variable configuration from stable transformation logic and creates one auditable place to change the value. Professional upgrade:

Use parameters for controlled environmental or run-time values, combine them with clear query dependencies, and retain a release log when parameter changes affect published outputs.

10. Common beginner mistakes

1. Calling a normal cell a parameter without connecting it to the query. | 2. Using parameters to hide a business rule that should be governed as data. | 3. Changing multiple parameters at once, making validation ambiguous. | 4. Storing credentials in plain workbook

cells.

11. If you get stuck - recovery ladder

Reset to the previous value, refresh, record baseline controls, then change one parameter only. Trace every query step that references it. If still stuck, use only the partial skeleton in the practice workbook; save the attempt before opening review.

12. Now you do a small version alone

Fresh case: create pRegion or pFolderPath with a documented contract and before/after validation.

13. Validate your result

Parameter purpose/type/owner recorded; only expected rows/files change; baseline can be restored reproducibly.

14. Teach it back in your own words

Explain the difference between configuration, business data, and a secret.

15. Protected review + changed fresh retry

Attempt first. Changed retry uses a different parameter type and expected effect.

16. Evidence / mastery + exact next route

Parameter contract + query reference + before/after controls + explain-back. After saving evidence, continue to AX10.

AX10 - Refreshable monthly reporting workflow

1. Why it matters

A green Refresh All message only proves the engine ran. A professional monthly process must prove the expected files arrived, the correct scope was used, key errors were controlled, totals reconciled, and the release was approved.

2. What you are learning

Run a month-end release checklist: intake, expected/actual files, backup exclusion, parameter values, refresh, row/file/key/error controls, critical totals, issue resolution, approval and archive.

3. Prerequisite / what you already know

AX1-AX9. This lesson integrates formula controls and Power Query controls.

4. Picture it in your head

Month-end conveyor: New files arrive -> scope check -> refresh -> exceptions -> reconcile -> management summary -> release log. A refreshable report is a repeatable operating process, not a workbook that happened to work once.

5. New words before we use them

Pre-check: Validation performed before refresh to confirm expected inputs and configuration. Release gate: A condition that must pass before a report is published. Rerun evidence: Saved proof that the process was executed again after a repair and that controls now pass.

6. Watch / visual source

RECOMMENDED VIDEO: Power Query - Beginner to PRO Masterclass in 30 minutes 33:35-35:17 refresh/load visualization; course book controls release discipline.

<https://www.youtube.com/watch?v=MMdcczmULrU>

After watching/opening: return to this lesson and reproduce the worked example from your own file/query/report. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Receive: place only expected source files in the intake folder or maintain a file whitelist. Record expected names and count.
- Pre-check: confirm schema, date coverage, parameter values and backup exclusion before Refresh All.
- Refresh: run queries. Save any error, not just the successful screen.
- Validate: file contributions, total rows, duplicates, unmatched keys, error rows, critical Net total and Pivot/report reconciliation.
- Approve: if all blocking controls pass, record release status and owner. If one critical control fails, do not publish.
- Repair and rerun: change one cause, rerun the full relevant controls, then publish and archive source/version/evidence.

8. What you should see / be able to say

A repeatable month-end run with a visible PASS/BLOCKED decision and evidence for every critical control.

9. Why it worked

The workflow treats refresh as one step inside a controlled release process rather than as the definition of success.

Professional upgrade: Treat recurring

spreadsheet/Power Query reporting as an operational process with intake controls, release criteria, issue log, evidence archive and handoff-not as a personal workbook ritual.

10. Common beginner mistakes

1. Publishing because Refresh All showed no error. | 2. Leaving backup files in the source folder. | 3. Changing a parameter and forgetting to record it. | 4. Fixing a failed source file without rerunning reconciliations.

11. If you get stuck - recovery ladder

Return to the release checklist in order: expected inputs -> configuration -> query errors -> row/key controls -> totals -> report outputs. Find the first failed control and repair from there. If still stuck, use only the partial skeleton in the practice workbook; save the attempt before opening review.

12. Now you do a small version alone

Fresh case: execute a May reporting cycle with one unexpected backup, one unmatched product and one corrected source file. Record block, repair and rerun evidence.

13. Validate your result

Expected/actual files match; no uncontrolled duplicate/unmatched error; critical total reconciles; release owner/status saved; rerun evidence present.

14. Teach it back in your own words

Explain why 'refresh succeeded' and 'report is safe to publish' are different statements.

15. Protected review + changed fresh retry

Attempt first. Changed retry uses a quarter-end folder with one schema drift and one late file.

16. Evidence / mastery + exact next route

Completed release checklist + error/recovery evidence + reconciliations + 90-second release defense. After saving evidence, complete the module mastery checklist and follow the Gate A / completion route.